



OBJECT-ORIENTED PROGRAMMING
PROJECT DOCUMENTATION

Hospital Patient & Clinic Appointment Scheduler

Topic Option: 14
Implementation Tier: Tier 1 — Console Application

Group Members

Meklit Kiros | Mebatsion Yoseph | HayleMariyam Agergzat | Hewan Kifle | Beka Assefa

Department of Electrical & Computer Engineering
June 2026

1. PROJECT IDENTITY

Field	Details
Project Title	Hospital Patient & Clinic Appointment Scheduler
Topic Option	14 — Hospital Patient & Clinic Appointment Scheduler
Implementation Tier	Tier 1: Console Application (Pure Java)
Programming Language	Java SE (JDK 8+)
IDE / Build Tool	Apache NetBeans 8.2 / Ant Build System
Package	hospitalsystem
Submission Date	June 2026

2. GROUP MEMBERS

Name	Student ID	Role
Meklit Kiros	UGR/0511/17	Lead Architect & Logic Developer
Mebatsion Yoseph	UGR/3659/17	UI/UX Designer
HayleMariyam Agergzat	UGR/9851/17	Backend Developer
Hewan Kifle	UGR/7673/17	QA Tester & Documentation
Beka Assefa	UGR/8958/17	Version Control & Integration Manager

3. PROBLEM STATEMENT

Modern healthcare facilities face significant administrative challenges in managing patient records, scheduling clinical appointments, and coordinating medical staff workflows. Manual or paper-based systems introduce errors in scheduling, lead to data inconsistencies, and make it difficult to track doctor-patient relationships efficiently.

This project addresses those challenges by developing a structured, object-oriented Java console application that models the core entities of a hospital environment — Patients, Doctors, Nurses, and the Hospital itself — and demonstrates how clean OOP design can serve as the architectural foundation for a real healthcare management system.

Target End-Users:

- Hospital administrative staff managing patient intake and appointment scheduling.
- Medical professionals (doctors, nurses) requiring organised patient interaction records.
- Software engineering students studying OOP patterns applied to real-world domains.

4. SYSTEM OVERVIEW & TIER SELECTION

Implementation Tier: Tier 1 — Console Application

The project is implemented as a pure Java console application with zero external framework dependencies. This tier was selected to maximise focus on OOP architecture quality, class relationship correctness, and algorithmic logic without the overhead of GUI or web frameworks.

Core Operational Features:

Person Modelling	An abstract Person class serves as the unified base for all human entities in the system.
Doctor Management	Doctors are registered with a specialisation area and a unique ID. Each doctor maintains an ArrayList of assigned nurses and can check patients.
Nurse Coordination	Nurses are assigned to doctors. A command/response mechanism simulates doctor-to-nurse delegation in real clinical workflows.
Patient Registration	Patients (Petiant) are registered with basic demographics. They can initiate interactions with doctors through association methods.
Hospital Composition	The Hospital class aggregates a Doctor and Nurse through a HAS-A composition relationship, modelling the real institutional structure.
Daily Routine Interface	The Daily interface enforces eat() and sleep() contracts across all person subclasses, demonstrating capability-based design (CAN-DO).
Polymorphic Dispatch	Person references are used to hold Doctor and Nurse objects, enabling runtime polymorphic method resolution on shared behaviours.

5. TECHNOLOGY STACK

Component	Technology	Purpose
Language	Java SE 8+	Core OOP implementation language
IDE	Apache NetBeans 8.2	Development environment & project scaffolding
Build System	Apache Ant (build.xml)	Compilation, packaging, JAR distribution
Runtime	JVM (Standard Edition)	Console execution environment
Data Structure	java.util.ArrayList	Dynamic nurse roster management in Doctor
Version Control	Git / GitHub	Source code management & collaboration
Distribution	JAR Archive	Executable standalone package (dist/ folder)

6. OOP ARCHITECTURAL DESIGN

6.1 Class Hierarchy Overview

The system is composed of 7 Java source files organised under the *hospitalsystem* package. The class hierarchy is anchored by an abstract **Person** superclass, which is extended by three concrete subclasses. A standalone **Hospital** class and a **Daily** interface complete the architecture.

Layer	Class / Interface	Type	Relationship
Root	Person	abstract class	Base superclass
Level 1	Doctor	concrete class	extends Person, implements Daily
Level 1	Nurse	concrete class	extends Person, implements Daily
Level 1	Patient	concrete class	extends Person, implements Daily
Container	Hospital	concrete class	Composition: HAS-A Doctor & Nurse
Contract	Daily	interface	CAN-DO contract: eat(), sleep()
Main	HospitalSystem	driver class	Entry point, demonstrates all OOP concepts

6.2 Encapsulation

All domain-sensitive fields across the codebase are declared **private** and accessed exclusively through public getter and setter methods. This prevents unauthorised direct modification of critical data such as employee IDs, specialisation areas, and nurse rank.

Key encapsulated fields:

Class	Private Field(s)	Access Method(s)
Person	name, age, sex	getName/setName, getAge/setAge, ...
Doctor	specializationArea, ID_num	getSpecializationArea(), getID_num()
Nurse	ID_num, stature, getCommand	getID_num(), getStature(), setCommand()
Hospital	doctors, nurses	getDoctors(), getNurses()

6.3 Inheritance

The IS-A inheritance chain flows from the abstract **Person** superclass down to three concrete domain subclasses. Each subclass inherits the common demographic attributes (name, age, sex) and their associated accessors, then extends the base with specialised fields and behaviours:

- **Doctor** adds specializationArea, ID_num, and an ArrayList of Nurse associations.
- **Nurse** adds ID_num, stature rank, and a command-receipt mechanism.
- **Patient** adds patient-specific interaction methods and doctor communication.

Constructors in all subclasses call **super(name, age, sex)** to honour the parent contract and avoid code duplication.

6.4 Polymorphism

Two forms of polymorphism are demonstrated:

Runtime (dynamic) polymorphism: The **vision()** method is declared in Person and overridden independently in Doctor, Nurse, and Patient. When called on a Person reference pointing to any subclass instance, the JVM resolves the correct overriding method at runtime:

```
Person Firaol = new Doctor("Firaol Terefe", 24, "Male", "Brain", 2134);  
Firaol.vision(); // dispatches to Doctor.vision() at runtime
```

Compile-time (static) polymorphism — method overloading: The Doctor class overloads the **eat()** method: one version accepts no arguments (general eating), and one accepts a **String foodtype** parameter, demonstrating structural overloading within a single class.

6.5 Abstraction

Abstraction is applied at two levels:

Abstract Class — Person: The Person class is declared abstract, meaning it cannot be instantiated directly. It defines the shared structure (name, age, sex) and a concrete vision() method that subclasses must override to provide domain-specific behaviour. This enforces a consistent interface across all human entities.

Interface — Daily: The Daily interface models the CAN-DO capability contract, mandating that any class implementing it must provide eat() and sleep() method bodies. Doctor, Nurse, and Patient all implement Daily, demonstrating that unrelated classes can share a capability contract without sharing an identity hierarchy.

7. CLASS-BY-CLASS DESCRIPTIONS

Person (abstract class)

Role: Abstract base for all human entities in the hospital system.

Fields: name : String, age : int, sex : String

Methods: getName/setName, getAge/setAge, getSex/setSex, vision() [overrideable]

OOP Significance: Abstraction anchor — cannot be instantiated; forces subclasses to specialise.

Doctor (extends Person, implements Daily)

Role: Represents a registered medical doctor with a clinical specialisation.

Private Fields: specializationArea : String, ID_num : int

Collections: ArrayList nurses — maintains an aggregated roster of assigned nurses.

Key Methods: commandNurse(Nurse), addNurse(Nurse), fireNurse(Nurse), checkPatient(Patient), listen(), eat(), eat(String), sleep(), vision()

OOP Significance: Demonstrates inheritance, interface implementation, aggregation (ArrayList), method overloading on eat(), and polymorphic vision() override.

Nurse (extends Person, implements Daily)

Role: Represents a registered nurse assigned under a supervising doctor.

Private Fields: ID_num : int, stature : String, getCommand : boolean

Key Methods: getCommanded(), eat(), sleep(), vision()

OOP Significance: Demonstrates inheritance and interface fulfilment; participates in aggregation relationship with Doctor.

Patient (extends Person, implements Daily)

Role: Represents a hospital patient registered for clinical consultation.

Key Methods: talkToDoctor(Doctor), beingChecked(), eat(), sleep(), vision()

OOP Significance: Association with Doctor class — uses Doctor's reference to invoke interaction without owning it (loose coupling).

Hospital (concrete class)

Role: Models the hospital institution as a container for its core staff.

Private Fields: doctors : Doctor, nurses : Nurse

Key Methods: getDoctors(), getNurses()

OOP Significance: Composition (HAS-A) — Hospital cannot function without its Doctor and Nurse; both are created externally and injected through the constructor.

Daily (interface)

Role: CAN-DO capability contract representing shared daily human activities.

Abstract Methods: void eat(), void sleep()

Implementing Classes: Doctor, Nurse, Patient

OOP Significance: Demonstrates the IS-A vs CAN-DO distinction from the project brief. Any class that CAN perform daily routines implements this interface regardless of its identity hierarchy.

HospitalSystem (driver / main class)

Role: Application entry point — demonstrates all OOP relationships through concrete instantiation.

Demonstrated Concepts: Object creation, composition (Hospital), aggregation (Doctor-Nurse list), association (Doctor-Patient), polymorphism (Person reference to Doctor), and interface-driven behaviour.

8. OOP RELATIONSHIP SUMMARY

Relationship Type	Between	Description
Inheritance (IS-A)	Doctor / Nurse / Patient → Person	Subclasses reuse and extend the Person blueprint.
Abstraction	Person (abstract class)	Cannot be instantiated; enforces structural contract on subclasses.
Interface (CAN-DO)	Doctor / Nurse / Patient → Daily	All three implement Daily's eat() and sleep() capability contract.
Composition (HAS-A)	Hospital → Doctor, Nurse	Hospital owns Doctor and Nurse; they are initialised together with the Hospital object.
Aggregation (HAS-A)	Doctor → ArrayList[Nurse]	Doctor maintains a list of Nurse objects; nurses can be added or removed independently.
Association (USES-A)	Doctor ↔ Patient	Doctor checks Patient; Patient talks to Doctor — neither owns the other.
Polymorphism	Person ref → Doctor instance	vision() is resolved at runtime to Doctor.vision() via dynamic dispatch.
Method Overloading	Doctor.eat() / Doctor.eat(String)	Two eat() signatures in Doctor demonstrate compile-time polymorphism.

9. ENGINEERING ACHIEVEMENTS

9.1 Clean Hierarchical Architecture

The project achieves a textbook-clean three-tier class hierarchy: a single abstract root (Person), three specialised leaf subclasses (Doctor, Nurse, Patient), and an orthogonal interface (Daily) layered across the hierarchy. This demonstrates the Liskov Substitution Principle — any Person-typed variable can hold a Doctor, Nurse, or Patient object and invoke shared methods without knowing the concrete type.

9.2 Dual-Form Polymorphism

The codebase implements both recognised forms of Java polymorphism. Runtime polymorphism is achieved through method overriding (vision() across all subclasses), while compile-time polymorphism is demonstrated through method overloading (eat() and eat(String) in Doctor). This dual demonstration directly addresses the project rubric requirement.

9.3 Three OOP Relationship Patterns

The system correctly implements and labels all three major inter-object relationship patterns: Composition (Hospital tightly owns Doctor and Nurse), Aggregation (Doctor loosely owns a dynamic Nurse roster via ArrayList), and Association (Doctor and Patient interact without ownership). This range of relationships demonstrates nuanced OOP design literacy.

9.4 Interface-Driven Design

The Daily interface illustrates the CAN-DO design principle highlighted in the project manual: Doctor, Nurse, and Patient implement Daily not because they share an identity, but because they share a capability. This separation of capability from identity is a hallmark of well-structured OOP systems.

9.5 Runnable Executable Distribution

The project is packaged as a standalone JAR executable (dist/HospitalSystem.jar) via the Ant build system, demonstrating full-cycle engineering from source authoring to distributable artifact. The application runs locally without any additional dependency installation.

10. TEAM RESPONSIBILITIES

Member	Role	Contributions
Meklit Kiros	Lead Architect & Logic Developer	Designed the overall class hierarchy, implemented the Person abstract class and Doctor class, architected all OOP relationships, and led main driver logic in HospitalSystem.java.
Mebatsion Yoseph	UI/UX Designer	Designed the console interaction flow, formatted all terminal output for readability, produced system architecture diagrams, and ensured a user-friendly runtime experience.
HayleMariyam Agergzat	Backend Developer	Implemented the Nurse, Patient, and Hospital classes, established aggregation and association relationship logic, and integrated all components into a cohesive backend structure.
Hewan Kifle	QA Tester & Documentation	Conducted all functional test runs, validated OOP correctness across classes, wrote in-code comments, and authored this project documentation.
Beka Assefa	Version Control & Integration Manager	Managed the GitHub repository, maintained commit history and branching strategy, resolved merge conflicts, and ensured all team contributions were cleanly integrated.

11. PROJECT TIMELINE

Week	Period	Milestone	Deliverable
Week 1	May 1–7	Project selection & planning	Topic chosen, roles assigned
Week 2	May 8–14	Phase 1 Proposal writing	PDF proposal submitted
Week 3	May 15–21	Core class hierarchy design	Person, Doctor, Nurse skeletons
Week 4	May 22–28	OOP relationships implementation	Composition, aggregation, association
Week 5	May 29–Jun 4	Patient class & interface Daily	Full class set complete
Week 6	Jun 5	Phase 1 Deadline	Proposal submitted on deadline
Week 7	Jun 6–12	Driver class & integration testing	HospitalSystem.java finalised
Week 8	Jun 13–19	Code review & refinement	Bug fixes, comment cleanup
Week 9	Jun 20–26	Documentation & final packaging	This document + JAR distribution

Week 10	Pre-Exams	Final submission & presentation	GitHub URL submitted, live demo ready
------------	-----------	---------------------------------	---------------------------------------

12. SOURCE CODE REPOSITORY

The complete project source code is hosted in a version-controlled GitHub repository. All Java source files, the NetBeans project structure, Ant build configuration, and the compiled JAR distribution are available at the link below.

Repository URL	https://github.com/[group-repo]/HospitalSystem
Main Branch	main
Source Directory	HospitalSystem/src/hospitalssystem/
Executable JAR	HospitalSystem/dist/HospitalSystem.jar
Build File	HospitalSystem/build.xml (Ant)

■ **NOTE:** As per the submission guidelines, source code is NOT included in this documentation. The full codebase is accessible only via the repository URL above.

13. EVALUATION CRITERIA ALIGNMENT

Criterion	Weight	How We Address It	Status
OOP Design	40%	All four pillars (Encapsulation, Inheritance, Polymorphism, Abstraction) demonstrated with dual polymorphism forms and 5 relationship types.	Complete
System Functionality	30%	Doctor-Patient-Nurse interactions, Hospital composition, nurse roster management all operational in console output.	Complete
Documentation	15%	This document covers all required sections: title page, problem statement, system overview, tech stack, team roles, timeline, and architectural design.	Complete
Presentation & Defense	15%	Application is fully runnable from JAR; all team members can explain every OOP decision during Q&A.	Ready

We, the undersigned group members, declare that this project was completed independently and that all design decisions, source code, and documentation are original work produced by our team. No source code or architectural logic was copied from third-party repositories without proper attribution.

Meklit Kiros

Mebatsion Yoseph

HayleMariyam Agergzat

Hewan Kifle

Beka Assefa